

R4DS 08 – Logicals & Numbers

Comparisons

Numeric Comparisons

Create logical vectors with <, <=, >, >=, !=, ==:

```
flights |>
  mutate(
    daytime = dep_time > 600 & dep_time < 2000,
    approx_ontime = abs(arr_delay) < 20,
    .keep = "used"
  )
```

```
#> # A tibble: 336,776 x 4
```

```
#>   dep_time arr_delay daytime approx_ontime
```

```
#>   <int>     <dbl> <lgl>    <lgl>
```

```
#> 1      517      11 FALSE    TRUE
```

```
#> 2      533      20 FALSE    FALSE
```

```
#> ... [output truncated]
```

Naming intermediate logical steps makes complex filters easier to read and debug.

Floating Point Comparison

Beware of == with computed numbers:

```
x <- c(1 / 49 * 49, sqrt(2) ^ 2)
x == c(1, 2)
```

```
#> [1] FALSE FALSE
```

```
print(x, digits = 16)
```

```
#> [1] 0.9999999999999999 2.0000000000000004
```

Use `dplyr::near()` to handle floating point issues:

```
near(x, c(1, 2))
```

```
#> [1] TRUE TRUE
```

Missing Values in Comparisons

`NA` is contagious — any operation involving `NA` returns `NA`:

```
NA > 5
```

```
#> [1] NA
```

```
10 == NA
```

```
#> [1] NA
```

```
NA == NA
```

```
#> [1] NA
```

Use `is.na()` to test for missing values:

```
is.na(c(TRUE, NA, FALSE))
```

```
#> [1] FALSE TRUE FALSE
```

```
is.na(c(1, NA, 3))
```

```
#> [1] FALSE TRUE FALSE
```

Using is.na() with Data

```
# Find cancelled flights
flights |> filter(is.na(dep_time))
```

```
#> # A tibble: 8,255 x 19
```

```
#>   year month   day dep_time sched_dep_time dep_delay
#>   <int> <int> <int>   <int>         <int>       <dbl>
#> 1  2013     1     1       NA           1630         NA
#> 2  2013     1     1       NA           1935         NA
#> ... [output truncated]
```

Sort NAs first with `arrange()`:

```
flights |>
  filter(month == 1, day == 1) |>
  arrange(desc(is.na(dep_time)), dep_time)
```

Exercises (Comparisons)

- 1 How does `dplyr::near()` work? Type `near` to see the source. Is `sqrt(2)^2` near 2?
- 2 Use `mutate()`, `is.na()`, and `count()` to describe how `dep_time`, `sched_dep_time`, and `dep_delay` missing values are connected.

Solutions (Comparisons)

① `near()` checks if `abs(x - y) < tol`, where `tol` defaults to `.Machine$double.eps^0.5`. Yes, `sqrt(2)^2` is near 2.

②

```
flights |>
  mutate(
    dep_na = is.na(dep_time),
    sched_na = is.na(sched_dep_time),
    delay_na = is.na(dep_delay)
  ) |>
  count(dep_na, sched_na, delay_na)
```

```
#> # A tibble: 2 x 4
#>   dep_na sched_na delay_na      n
#>   <lgl>   <lgl>    <lgl>   <int>
#> 1 FALSE  FALSE   FALSE  328521
#> 2 TRUE   FALSE   TRUE   8255
#> ... [output truncated]
```

When `dep_time` is NA, `dep_delay` is also NA. `sched_dep_time` is never missing.

Boolean Algebra

Boolean Operators

```
# & = AND, | = OR, ! = NOT, xor() = exclusive or
df |> filter(!is.na(x))          # x is not missing
df |> filter(x < -10 | x > 0)    # x outside [-10, 0]
```

NAs in Boolean algebra follow logic rules:

```
df <- tibble(x = c(TRUE, FALSE, NA))
df |> mutate(and = x & NA, or = x | NA)
```

```
#> # A tibble: 3 x 3
#>   x      and    or
#>   <lgl> <lgl> <lgl>
#> 1 TRUE  NA     TRUE
#> 2 FALSE FALSE  NA
#> 3 NA    NA     NA
#> ... [output truncated]
```

TRUE | NA is TRUE (at least one is TRUE). FALSE & NA is FALSE (at least one is FALSE).

Order of Operations and %in%

Common mistake — this does NOT filter Nov/Dec:

```
flights |> filter(month == 11 | 12) # WRONG!  
# R evaluates: (month == 11) | 12 → always TRUE
```

Correct approaches:

```
flights |> filter(month == 11 | month == 12)  
flights |> filter(month %in% c(11, 12)) # preferred
```

%in% handles NAs differently from ==:

```
c(1, 2, NA) == NA
```

```
#> [1] NA NA NA
```

```
c(1, 2, NA) %in% NA
```

```
#> [1] FALSE FALSE TRUE
```

- 1 Find all flights where `arr_delay` is missing but `dep_delay` is not. Find all flights where neither `arr_time` nor `sched_arr_time` are missing, but `arr_delay` is.
- 2 How many flights have a missing `dep_time`? What other variables are missing? What might these rows represent?
- 3 Assuming missing `dep_time` = cancelled, look at cancelled flights per day. Is there a connection between proportion cancelled and average delay?

❶

```
flights |>
  filter(is.na(arr_delay) & !is.na(dep_delay)) |>
  nrow()
```

```
#> [1] 1175
```

```
flights |>
  filter(!is.na(arr_time) & !is.na(sched_arr_time) &
         is.na(arr_delay)) |>
  nrow()
```

```
#> [1] 717
```

- ❷ 8,255 flights. `dep_delay`, `arr_time`, `arr_delay`, `air_time` also missing — these are cancelled flights.
- ❸ Higher average delay days tend to have higher cancellation proportions.

Logical Summaries

any() and all()

```
flights |>
  group_by(year, month, day) |>
  summarize(
    all_delayed = all(dep_delay <= 60, na.rm = TRUE),
    any_long_delay = any(arr_delay >= 300, na.rm = TRUE),
    .groups = "drop"
  )
```

```
#> # A tibble: 365 x 5
#>   year month   day all_delayed any_long_delay
#>   <int> <int> <int> <lgl>         <lgl>
#> 1  2013     1     1 FALSE         TRUE
#> 2  2013     1     2 FALSE         TRUE
#> ... [output truncated]
```

`any()` = equivalent of `|`. `all()` = equivalent of `&`.

Numeric Summaries of Logicals

TRUE becomes 1, **FALSE** becomes 0 in numeric context. So `sum()` counts TRUEs, `mean()` gives proportion:

```
flights |>
  group_by(year, month, day) |>
  summarize(
    prop_delayed = mean(dep_delay <= 60, na.rm = TRUE),
    count_long = sum(arr_delay >= 300, na.rm = TRUE),
    .groups = "drop"
  )
```

```
#> # A tibble: 365 x 5
#>   year month   day prop_delayed count_long
#>   <int> <int> <int>         <dbl>         <int>
#> 1  2013     1     1         0.939             3
#> 2  2013     1     2         0.914             3
#> ... [output truncated]
```


Logical Subsetting with [

Use `[` to filter within `summarize()` — compute different summaries for different subsets:

```
flights |>
  group_by(year, month, day) |>
  summarize(
    behind = mean(arr_delay[arr_delay > 0], na.rm = TRUE),
    ahead = mean(arr_delay[arr_delay < 0], na.rm = TRUE),
    n = n(),
    .groups = "drop"
  )
```

```
#> # A tibble: 365 x 6
#>   year month   day behind ahead    n
#>   <int> <int> <int>   <dbl> <dbl> <int>
#> 1  2013     1     1   32.5 -12.5   842
#> 2  2013     1     2   32.0 -14.3   943
#> ... [output truncated]
```

- ❶ What will `sum(is.na(x))` tell you? How about `mean(is.na(x))`?
- ❷ What does `prod()` return on a logical vector? What logical summary is it equivalent to? What about `min()`?

Solutions (Logical Summaries)

- ① `sum(is.na(x))` = count of NAs. `mean(is.na(x))` = proportion of NAs.
- ② `prod()` on logicals: returns 1 only if all are TRUE (equivalent to `all()`). Returns 0 if any is FALSE. `min()` returns 0 if any is FALSE, 1 if all TRUE — also equivalent to `all()`.

```
x <- c(TRUE, TRUE, FALSE)
prod(x)
```

```
#> [1] 0
```

```
min(x)
```

```
#> [1] 0
```

Conditional Transformations

if_else()

Three required arguments: condition, value if TRUE, value if FALSE:

```
x <- c(-3:3, NA)
if_else(x > 0, "+ve", "-ve")
```

```
#> [1] "-ve" "-ve" "-ve" "-ve" "+ve" "+ve" "+ve" NA
```

Optional fourth argument for NAs:

```
if_else(x > 0, "+ve", "-ve", "???)
```

```
#> [1] "-ve" "-ve" "-ve" "-ve" "+ve" "+ve" "+ve" "???"
```

Vectorized — can use variables on both sides:

```
if_else(x < 0, -x, x) # simple abs()
```

```
#> [1] 3 2 1 0 1 2 3 NA
```

case_when()

For multiple conditions, use `case_when()` with `condition ~ output` pairs:

```
case_when(  
  x == 0 ~ "0",  
  x < 0 ~ "-ve",  
  x > 0 ~ "+ve",  
  is.na(x) ~ "???"  
)
```

```
#> [1] "-ve" "-ve" "-ve" "0" "+ve" "+ve" "+ve" "???"
```

Use `.default` for catch-all:

```
case_when(  
  x < 0 ~ "-ve",  
  x > 0 ~ "+ve",  
  .default = "???"  
)
```

case_when() with Data

```
flights |>
  mutate(
    status = case_when(
      is.na(arr_delay) ~ "cancelled",
      arr_delay < -30 ~ "very early",
      arr_delay < -15 ~ "early",
      abs(arr_delay) <= 15 ~ "on time",
      arr_delay < 60 ~ "late",
      arr_delay < Inf ~ "very late",
    ),
    .keep = "used"
  )
```

```
#> # A tibble: 336,776 x 2
```

```
#>   arr_delay status
```

```
#>   <dbl> <chr>
```

```
#> 1      11 on time
```

```
#> 2      20 late
```

```
#> 3      33 late
```

```
#> ... [output truncated]
```

First matching condition wins. Unmatched rows get **NA**.

Exercises (Conditional)

- 1 Use `if_else()` to determine whether each number 0–20 is even or odd. (Hint: `x %% 2 == 0`.)
- 2 Given days like `c("Monday", "Saturday", "Wednesday")`, label as weekends or weekdays.
- 3 Use `if_else()` to compute absolute value of a numeric vector.
- 4 Write a `case_when()` using `month` and `day` from `flights` to label US holidays.

1

```
x <- 0:20  
if_else(x %% 2 == 0, "even", "odd")
```

```
#> [1] "even" "odd"  "even" "odd"  "even" "odd"  "even"  
#> [8] "odd"  "even" "odd"  "even" "odd"  "even" "odd"  
#> [15] "even" "odd"  "even" "odd"  "even" "odd"  "even"
```

2

```
x <- c("Monday", "Saturday", "Wednesday")  
if_else(x %in% c("Saturday", "Sunday"),  
        "weekend", "weekday")
```

```
#> [1] "weekday" "weekend" "weekday"
```

3 if_else(x < 0, -x, x)

```
flights |>
  mutate(
    holiday = case_when(
      month == 1 & day == 1 ~ "New Year's Day",
      month == 7 & day == 4 ~ "Independence Day",
      month == 12 & day == 25 ~ "Christmas",
      .default = NA
    ),
    .keep = "used"
  ) |>
  filter(!is.na(holiday))
```

```
#> # A tibble: 2,298 x 3
#>   month   day holiday
#>   <int> <int> <chr>
#> 1     1     1 1 New Year's Day
#> 2     1     1 1 New Year's Day
#> 3     1     1 1 New Year's Day
#> ... [output truncated]
```

Making Numbers & Counts

Parsing Strings to Numbers

```
x <- c("1.2", "5.6", "1e3")  
parse_double(x)
```

```
#> [1] 1.2 5.6 1000.0
```

`parse_number()` ignores non-numeric text:

```
x <- c("$1,234", "USD 3,513", "59%")  
parse_number(x)
```

```
#> [1] 1234 3513 59
```

count() and Variants

```
flights |> count(dest, sort = TRUE)
```

```
#> # A tibble: 105 x 2  
#>   dest      n  
#>   <chr> <int>  
#> 1 ORD    17283  
#> 2 ATL    17215  
#> ... [output truncated]
```

Equivalent with `group_by()` + `summarize()`:

```
flights |>  
  group_by(dest) |>  
  summarize(n = n(),  
            delay = mean(arr_delay, na.rm = TRUE))
```

```
#> # A tibble: 105 x 3  
#>   dest      n delay  
#>   <chr> <int> <dbl>  
#> 1 ABQ     254  4.38  
#> 2 ACK     265  4.85  
#> ... [output truncated]
```

Count Variants

```
# n_distinct: unique values per group
flights |>
  group_by(dest) |>
  summarize(carriers = n_distinct(carrier)) |>
  arrange(desc(carriers))
```

```
#> # A tibble: 105 x 2
#>   dest carriers
#>   <chr>   <int>
#> 1 ATL         7
#> 2 BOS         7
#> ... [output truncated]
```

```
# Weighted count
flights |> count(tailnum, wt = distance, sort = TRUE)
```

```
#> # A tibble: 4,044 x 2
#>   tailnum      n
#>   <chr>   <dbl>
#> 1 <NA> 1784167
#> ... [output truncated]
```

- ① How can you use `count()` to count rows with a missing value for a given variable?
- ② Expand to `group_by()` + `summarize()` + `arrange()`:
 - `flights |> count(dest, sort = TRUE)`
 - `flights |> count(tailnum, wt = distance)`

Solutions (Counts)

❶ `flights |> filter(is.na(dep_time)) |> count()` or `flights |> count(is.na(dep_time))`.

❷

```
# (a)
flights |> group_by(dest) |>
  summarize(n = n()) |> arrange(desc(n))

# (b)
flights |> group_by(tailnum) |>
  summarize(n = sum(distance))
```


Numeric Transformations

Recycling Rules

R recycles shorter vectors to match longer ones:

```
x <- c(1, 2, 10, 20)
x / 5          # recycles 5 to c(5, 5, 5, 5)
```

```
#> [1] 0.2 0.4 2.0 4.0
```

```
x * c(1, 2)    # recycles to c(1, 2, 1, 2)
```

```
#> [1] 1 4 10 40
```

Danger: silent wrong results if lengths don't match as expected:

```
# WRONG: recycles c(1, 2) row by row
flights |> filter(month == c(1, 2))
# RIGHT: use %in%
flights |> filter(month %in% c(1, 2))
```

pmin() and pmax()

Parallel (element-wise) min/max — different from `min()`/`max()` which return a single value:

```
df <- tribble(
  ~x, ~y,
  1, 3,
  5, 2,
  7, NA,
)
df |> mutate(min = pmin(x, y, na.rm = TRUE),
             max = pmax(x, y, na.rm = TRUE))
```

```
#> # A tibble: 3 x 4
#>       x     y   min   max
#>   <dbl> <dbl> <dbl> <dbl>
#> 1     1     3     1     3
#> 2     5     2     2     5
#> 3     7    NA     7     7
#> ... [output truncated]
```

Modular Arithmetic

$\%/\%$ = integer division, $\%\%$ = remainder:

```
1:10 %/% 3
```

```
#> [1] 0 0 1 1 1 2 2 2 3 3
```

```
1:10 %% 3
```

```
#> [1] 1 2 0 1 2 0 1 2 0 1
```

Useful for unpacking time variables:

```
flights |>
  mutate(hour = sched_dep_time %/% 100,
         minute = sched_dep_time %% 100,
         .keep = "used")
```

```
#> # A tibble: 336,776 x 3
```

```
#>   sched_dep_time  hour minute
```

```
#>           <int> <dbl>  <dbl>
```

```
#> 1           515     5     15
```

```
#> 2           529     5     29
```

```
#> ... [output truncated]
```

Rounding

```
round(123.456, 2) # nearest 0.01
```

```
#> [1] 123.46
```

```
round(123.456, 1) # nearest 0.1
```

```
#> [1] 123.5
```

```
round(123.456, -1) # nearest 10
```

```
#> [1] 120
```

Banker's rounding (round half to even):

```
round(c(1.5, 2.5))
```

```
#> [1] 2 2
```

`floor()` always rounds down, `ceiling()` always rounds up.

```
floor(123.456)
```

```
#> [1] 123
```

```
ceiling(123.456)
```

```
#> [1] 124
```

Logarithms and cut()

Three log functions: `log()` (base e), `log2()` (base 2), `log10()` (base 10).
Prefer `log2()` (doubling = +1) or `log10()` (easy back-transform).

`cut()` bins a numeric vector:

```
x <- c(1, 2, 5, 10, 15, 20)
cut(x, breaks = c(0, 5, 10, 15, 20),
     labels = c("sm", "md", "lg", "xl"))
```

```
#> [1] sm sm sm md lg xl
```

```
#> Levels: sm md lg xl
```

Values outside breaks become `NA`.

Cumulative Aggregates

```
x <- 1:10  
cumsum(x)
```

```
#> [1] 1 3 6 10 15 21 28 36 45 55
```

Also: `cumprod()`, `cummin()`, `cummax()`, `cummean()` (dplyr).

Useful for creating session IDs from gaps:

```
events |>  
  mutate(diff = time - lag(time, default = first(time)),  
         has_gap = diff >= 5,  
         group = cumsum(has_gap))
```

Exercises (Numeric Transformations)

- 1 Explain what each line does in computing proportion of cancelled flights by hour.
- 2 Convert `dep_time` and `sched_dep_time` to minutes since midnight.
- 3 Round `dep_time` and `arr_time` to the nearest five minutes.

Solutions (Numeric Transformations)

- 1 Group by hour (extracted via `dep_time %/% 100`), compute `mean(is.na(dep_time))` for proportion cancelled, count `n()`, filter small groups, plot.

2

```
flights |>
  mutate(
    dep_minutes = (dep_time %/% 100) * 60 +
                  dep_time %/% 100,
    .keep = "used"
  )
```

```
#> # A tibble: 336,776 x 2
#>   dep_time dep_minutes
#>   <int>     <dbl>
#> 1     517         317
#> 2     533         333
#> ... [output truncated]
```

- 3 `round(dep_time / 5) * 5`

General Transformations

Ranks

```
x <- c(1, 5, 5, 17, 22, NA)
min_rank(x)
```

```
#> [1] 1 2 2 4 5 NA
```

```
min_rank(desc(x))
```

```
#> [1] 5 3 3 2 1 NA
```

Variants: `row_number()` (no ties), `dense_rank()` (no gaps), `percent_rank()`, `cume_dist()`.

```
tibble(x) |>
  mutate(row_number = row_number(x),
         dense_rank = dense_rank(x),
         percent_rank = percent_rank(x))
```

```
#> # A tibble: 6 x 4
```

```
#>       x row_number dense_rank percent_rank
#>   <dbl>     <int>     <int>         <dbl>
#> 1     1         1         1           0
#> 2     5         2         2          0.25
#> 3     5         3         2          0.25
#> 4    17         4         3          0.75
#> 5    22         5         4           1
```

```
#> [context, transparency]
```

Offsets: lead() and lag()

```
x <- c(2, 5, 11, 11, 19, 35)
```

```
lag(x)
```

```
#> [1] NA  2  5 11 11 19
```

```
lead(x)
```

```
#> [1]  5 11 11 19 35 NA
```

Compute differences and detect changes:

```
x - lag(x)
```

```
#> [1] NA  3  6  0  8 16
```

```
x == lag(x)
```

```
#> [1]    NA FALSE FALSE  TRUE FALSE FALSE
```

Exercises (General Transformations)

- 1 Find the 10 most delayed flights using a ranking function. How do you handle ties?
- 2 Which plane (`tailnum`) has the worst on-time record?
- 3 What time of day should you fly to avoid delays?
- 4 For each destination, compute total delay minutes. For each flight, compute its proportion of destination's total delay.

Solutions (General) — 1-2

1

```
flights |>
  mutate(rank = min_rank(desc(arr_delay))) |>
  filter(rank <= 10) |>
  select(arr_delay, rank, carrier, flight) |>
  arrange(rank)
```

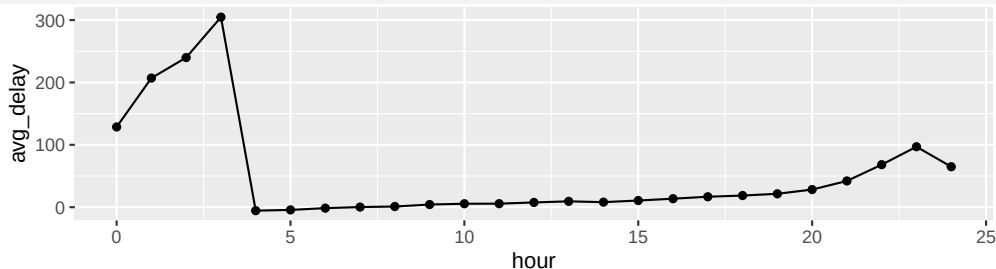
```
#> # A tibble: 10 x 4
#>   arr_delay rank carrier flight
#>   <dbl> <int> <chr>    <int>
#> 1     1272     1 HA         51
#> 2     1127     2 MQ        3535
#> ... [output truncated]
```

2

```
flights |>
  group_by(tailnum) |>
  summarize(prop_late = mean(arr_delay > 0, na.rm = TRUE),
            n = n()) |>
  filter(n >= 20) |>
  arrange(desc(prop_late))
```

3

```
flights |>
  group_by(hour = dep_time %/% 100) |>
  summarize(avg_delay = mean(dep_delay, na.rm = TRUE)) |>
  filter(!is.na(hour)) |>
  ggplot(aes(x = hour, y = avg_delay)) +
  geom_line() + geom_point()
```



Fly early morning (5–7 AM) for lowest delays.

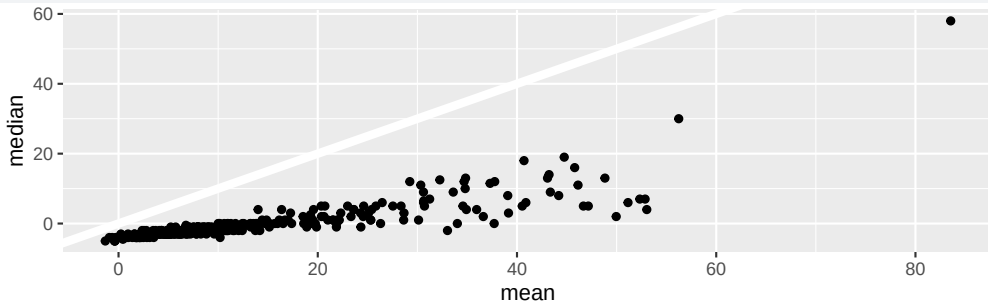
```
flights |>
  filter(arr_delay > 0) |>
  group_by(dest) |>
  mutate(
    total_delay = sum(arr_delay),
    prop_delay = arr_delay / total_delay
  ) |>
  select(dest, flight, arr_delay, prop_delay) |>
  ungroup()
```

```
#> # A tibble: 133,004 x 4
#>   dest  flight arr_delay prop_delay
#>   <chr>  <int>     <dbl>     <dbl>
#> 1 IAH      1545         11  0.000111
#> 2 IAH      1714         20  0.000201
#> ... [output truncated]
```


Numeric Summaries

Center: mean() vs. median()

```
flights |>  
  group_by(year, month, day) |>  
  summarize(mean = mean(dep_delay, na.rm = TRUE),  
            median = median(dep_delay, na.rm = TRUE),  
            .groups = "drop") |>  
  ggplot(aes(x = mean, y = median)) +  
  geom_abline(slope = 1, intercept = 0,  
            color = "white", linewidth = 2) +  
  geom_point()
```



Median always $\leq$ mean (right-skewed data).

Quantiles, Spread, and Positions

Quantiles: generalization of median.

```
flights |>
  group_by(year, month, day) |>
  summarize(
    max = max(dep_delay, na.rm = TRUE),
    q95 = quantile(dep_delay, 0.95, na.rm = TRUE),
    .groups = "drop"
  )
```

```
#> # A tibble: 365 x 5
#>   year month   day   max   q95
#>   <int> <int> <int> <dbl> <dbl>
#> 1  2013     1     1   853  70.1
#> 2  2013     1     2   379   85
#> ... [output truncated]
```

Spread: `sd()` and `IQR()` (middle 50%).

Positions: `first()`, `last()`, `nth(x, n)`.

Summaries with mutate()

Summary functions + `mutate()` = group standardization:

```
# Proportion of total
```

```
x / sum(x)
```

```
# Z-score
```

```
(x - mean(x)) / sd(x)
```

```
# Normalize to [0, 1]
```

```
(x - min(x)) / (max(x) - min(x))
```

```
# Index relative to first observation
```

```
x / first(x)
```

- 1 Brainstorm 5+ ways to assess typical delay characteristics. When is `mean()` useful? `median()`?
- 2 Which destinations show the greatest variation in air speed?

Solutions (Numeric Summaries)

- 1 Mean (symmetric data), median (skewed), proportion delayed, number > 30 min late, IQR of delays, 90th percentile. Use arrival delay for passenger impact, departure delay for operations. Join `planes` for aircraft age/type effects.

2

```
flights |>
  mutate(speed = distance / (air_time / 60)) |>
  group_by(dest) |>
  summarize(speed_sd = sd(speed, na.rm = TRUE),
            n = n()) |>
  filter(n >= 20) |>
  arrange(desc(speed_sd))
```

```
#> # A tibble: 97 x 3
#>   dest  speed_sd    n
#>   <chr>    <dbl> <int>
#> 1 OKC      38.3   346
#> 2 TUL      37.5   315
#> ... [output truncated]
```